Universidad Nacional de San Martín

TP Integrador Base de Datos

Materia	Base de Datos
Carrera	Tecnicatura en Programación Informática
Docentes	Gabriel FLORES - Rodrigo CASCO
Proyecto	Alero: sistema de reservas y reseñas para alojamientos
Integrantes	(completar)
Fecha de entrega	6 de noviembre de 2026
Repositorio	https://github.com/ju4nchp/tp-integrador-bd-alero
Aplicación publicada	https://alero-tp-bd.onrender.com

Índice

1. Descripción del problema
2. Descripción y origen del dataset
3. Análisis inicial de los datos
4. Proceso de normalización
5. Diagrama Entidad-Relación
6. Modelo relacional
7. Diseño de la solución NoSQL
8. Justificación de la utilización de NoSQL
9. Arquitectura general de la aplicación
10. Explicación del backend polimórfico
11. Tecnologías utilizadas
12. Instrucciones para ejecutar el proyecto
13. Ejemplos de funcionamiento
14. Comparación entre ambas implementaciones
15. Problemas encontrados y decisiones tomadas
16. Conclusiones

1. Descripción del problema

Elegimos el dominio de las reservas de viajes. Un alojamiento (un hotel, un hostel o unas cabañas) recibe reservas, algunas se cancelan y, cuando la estadía termina, el huésped puede dejar una reseña. El sistema lo usa el dueño o el encargado del alojamiento, no el huésped.

El sistema tiene que permitir:

- Cargar, consultar, modificar y dar de baja reservas.
- Cancelar una reserva indicando el motivo.
- Cargar la reseña de una estadía terminada, responderla y ponerle etiquetas.
- Ver estadísticas: facturación y cancelaciones por país, y reservas y puntajes por alojamiento.

Además definimos estas reglas:

- Las reservas no se borran. Darlas de baja las saca de los listados y de las estadísticas, pero siguen guardadas y se pueden ver en un historial (baja lógica).
- Solo se pueden modificar, cancelar o dar de baja las reservas pendientes o confirmadas.
- Las reseñas son solo de estadías completadas, una por reserva, y no se editan ni se borran. El alojamiento puede responderlas y etiquetarlas.

El objetivo del trabajo es diseñar el modelo relacional a partir de los datos, proponer una parte del sistema en una base NoSQL y armar una aplicación que trabaje con los dos motores.

2. Descripción y origen del dataset

Usamos el archivo **Travel_And_Tourism.csv**. Origen: (completar con el link de donde lo descargamos).

Cada fila es una reserva y trae todo junto: los datos del cliente, el destino, el hotel, el viaje, el pago, el estado y, si corresponde, la reseña o el motivo de cancelación.

Dato	**Cantidad**
Filas (reservas)	1000
Columnas	29
Clientes distintos	1000 (cada cliente tiene una sola reserva)
Hoteles	78
Ciudades de destino	26, en 10 países
Ciudades en total (clientes y destinos)	44, en 14 países
Estados de las reservas	772 completadas, 141 canceladas, 79 confirmadas y 8 pendientes

Grupo	Columnas del CSV
Reserva	Booking_ID, Booking_Date, Travel_Date, Return_Date, Booking_Status
Cliente	Customer_ID, Customer_Name, Customer_Age, Gender, Customer_City, Customer_Country
Destino y hotel	Destination_City, Destination_Country, Hotel_Name, Hotel_Rating
Viaje	Number_of_Travellers, Number_of_Adults, Number_of_Children, Number_of_Rooms, Number_of_Nights, Meal_Plan, Transportation_Type
Pago	Payment_Method, Discount_Amount, Total_Trip_Cost
Cancelación	Cancellation_Status, Cancellation_Reason
Reseña	Customer_Rating, Customer_Review_Comment

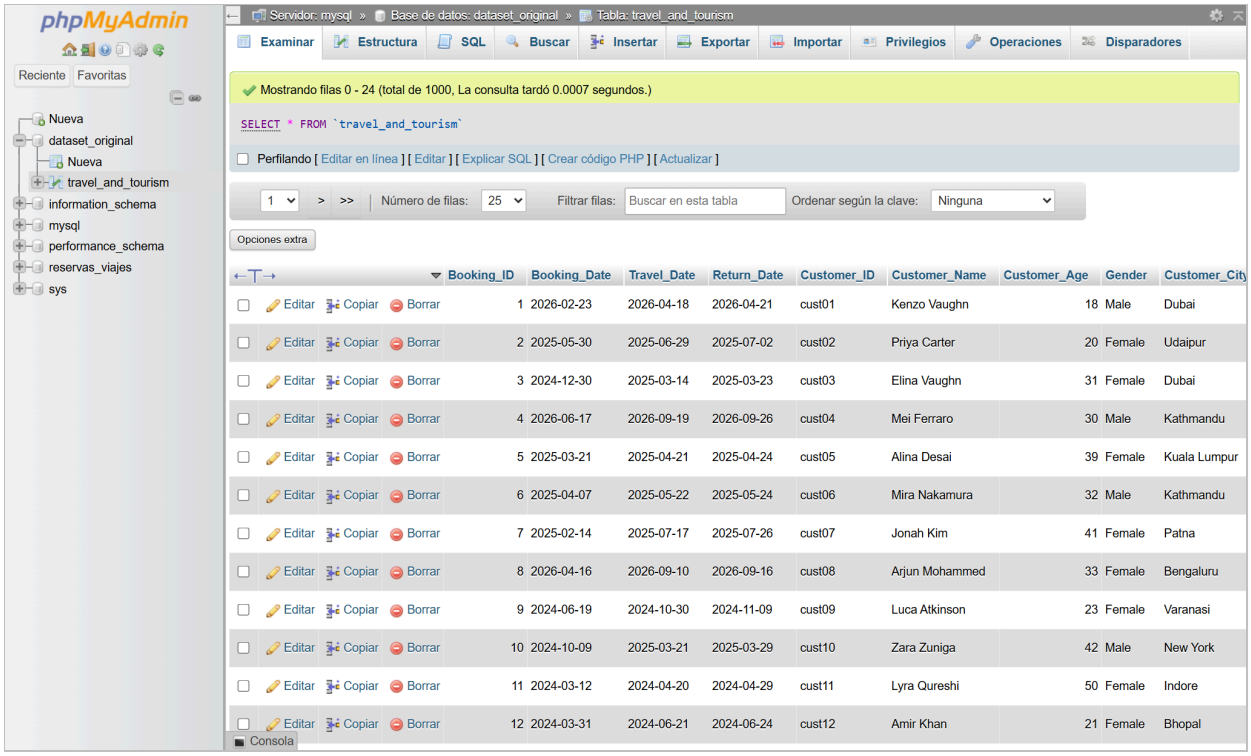


Figura 1. El CSV cargado tal cual en MySQL: una sola tabla de 29 columnas, una fila por reserva.

3. Análisis inicial de los datos

Antes de diseñar el modelo cargamos el CSV tal como viene en una tabla de MySQL (base `dataset_original`, script `database/analisis_dataset.sql`) y lo revisamos con consultas desde phpMyAdmin. Esto es lo que encontramos.

3.1 Datos repetidos

Los datos del hotel (nombre, ciudad y país) se repiten en cada reserva. Cedar Heights, por ejemplo, aparece en 25 filas con los mismos datos. Con la ciudad y el país del cliente pasa lo mismo.

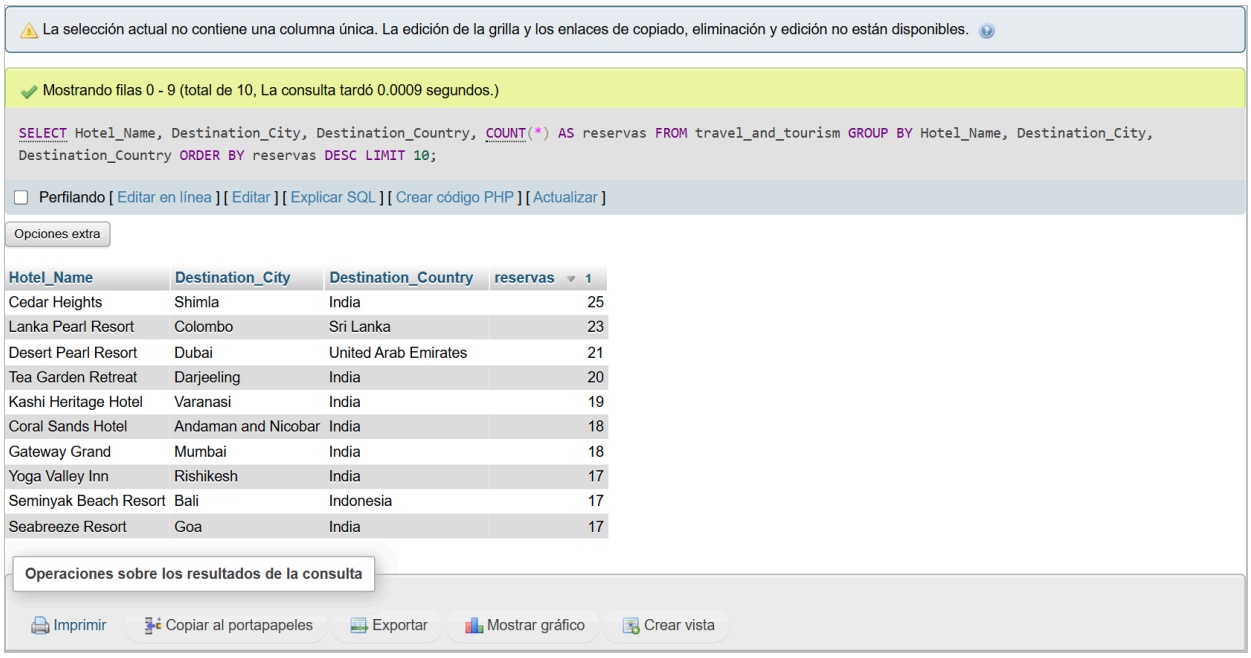


Figura 2. Cada hotel aparece muchas veces con su ciudad y su país repetidos.

3.2 Dependencias entre columnas

- `Customer_ID` determina el nombre, la edad, el género y la ciudad del cliente.
- `Hotel_Name` determina la ciudad del hotel.
- La ciudad determina el país: cada una de las 26 ciudades de destino tiene un solo país. Con las ciudades de los clientes pasa lo mismo.

Las dos últimas son dependencias transitivas: la reserva determina el hotel, el hotel la ciudad y la ciudad el país. Son las que hacen que el CSV no esté en tercera forma normal.

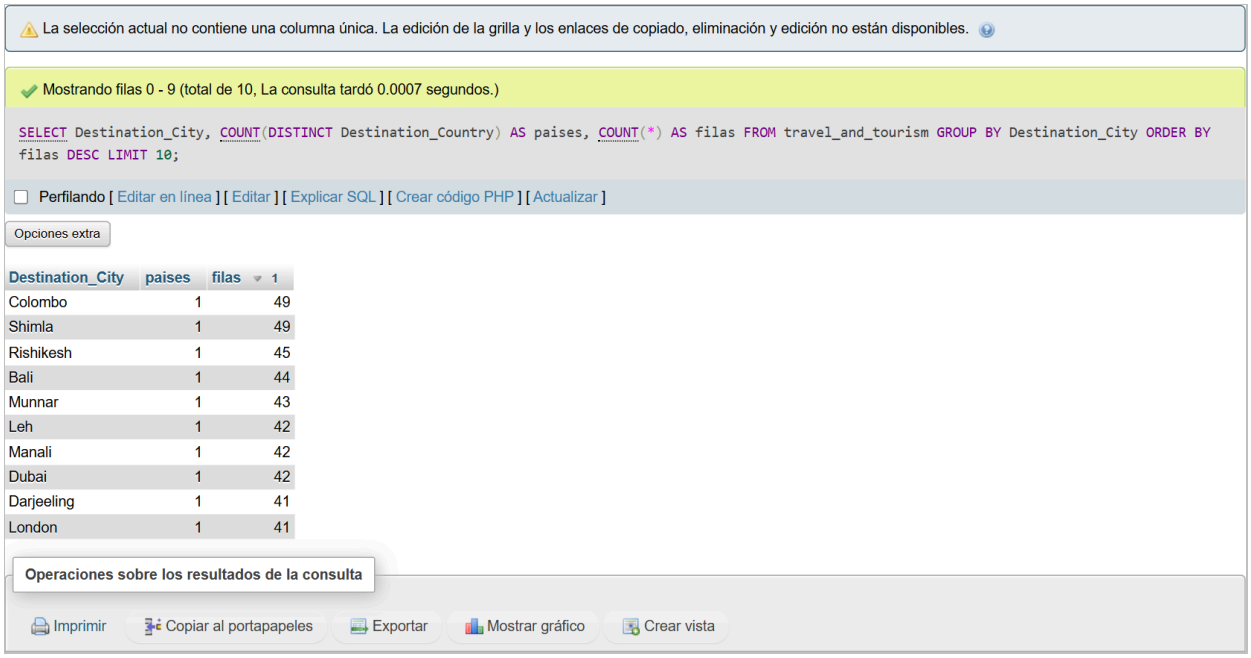


Figura 3. Ninguna ciudad tiene más de un país (columna `paises` en 1).

3.3 Columnas que se pueden calcular y valores vacíos disfrazados

- `Number_of_Travellers` es `Number_of_Adults + Number_of_Children`.
- `Number_of_Nights` es la diferencia entre `Return_Date` y `Travel_Date`.
- `Cancellation_Status` sale de `Booking_Status`: es "Cancelled" solo cuando la reserva está cancelada.

Lo comprobamos con una consulta y las tres coinciden en las 1000 filas, así que no hace falta guardarlas. La misma consulta muestra los valores vacíos disfrazados: `Cancellation_Reason` dice "Not Applicable" en las 859 reservas que no se cancelaron, y `Customer_Rating` vale 0 en las 228 que no tienen reseña. La cancelación y la reseña no existen en todas las reservas: son datos opcionales.

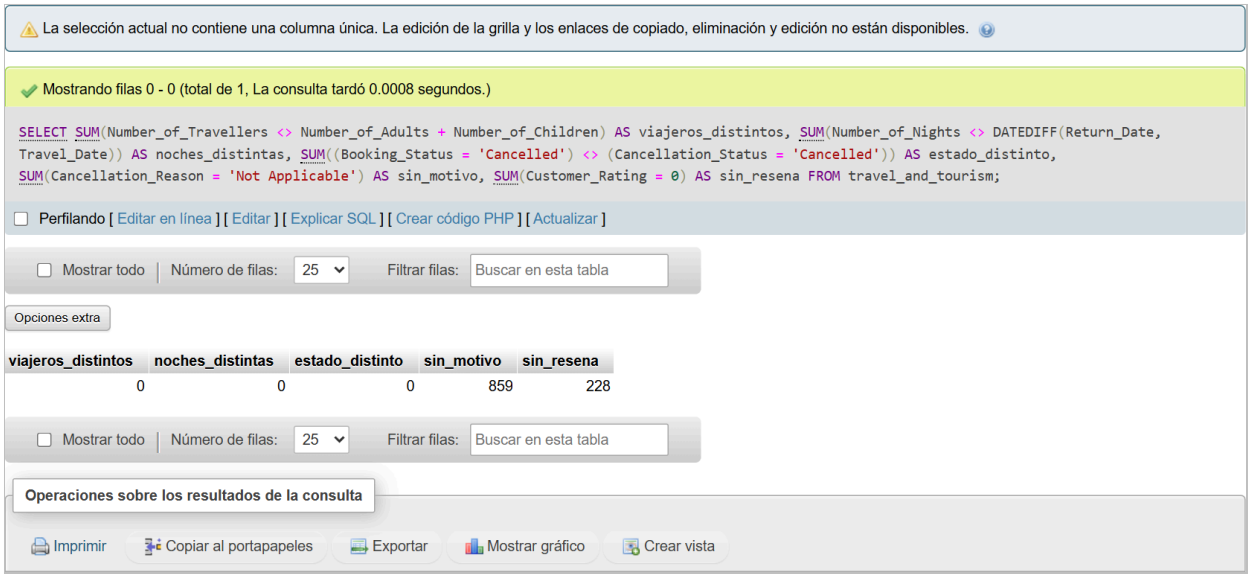


Figura 4. Las columnas calculadas no tienen diferencias (0) y hay 859 motivos "Not Applicable" y 228 puntajes en 0.

3.4 Un dato que no es del hotel

`Hotel_Rating` cambia en cada reserva aunque el hotel sea el mismo: Cedar Heights tiene puntajes de 1.0 a 5.0. No es una característica fija del hotel, es el puntaje que le puso el cliente en esa estadía. Por eso lo guardamos en la reseña (`puntaje_hotel`), y para el hotel calculamos una categoría base con el promedio.

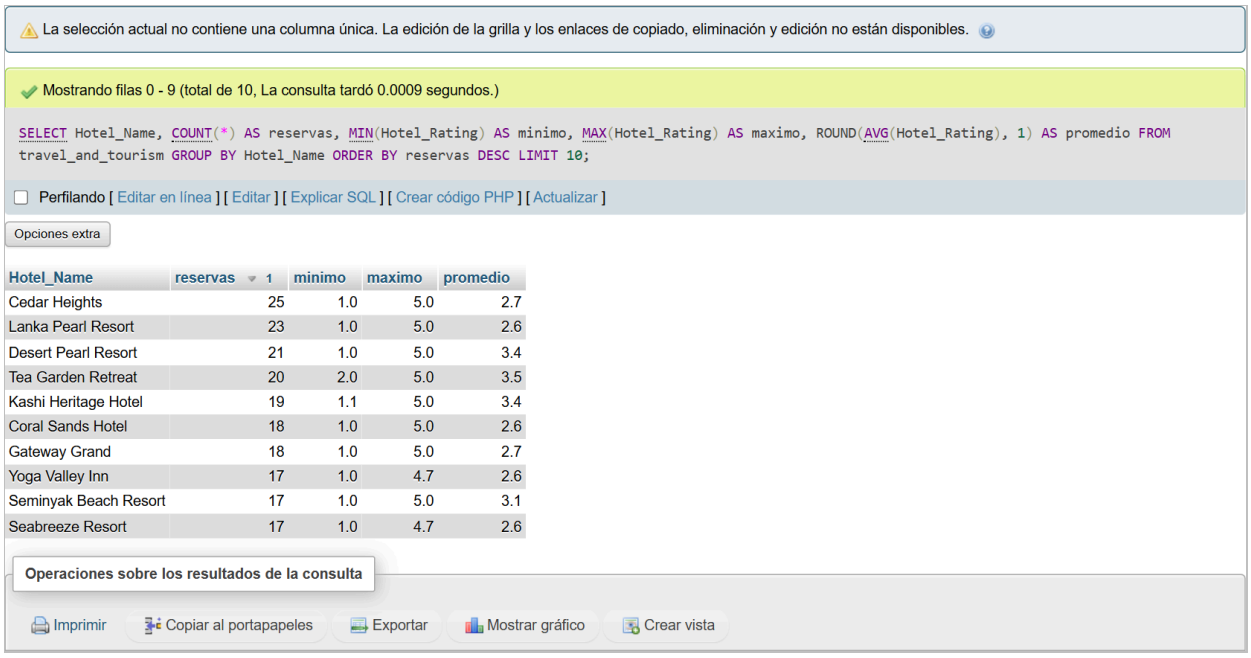


Figura 5. El puntaje del mismo hotel va de 1.0 a 5.0 según la reserva.

3.5 Problemas de tener todo en una sola tabla

- **Actualización:** si un hotel cambia de nombre hay que modificar todas sus filas (25 en el caso de Cedar Heights). Si se olvida alguna, el mismo hotel queda con dos nombres.
- **Inserción:** no se puede cargar un hotel o un cliente nuevo si no tiene una reserva.
- **Eliminación:** si se borra la única reserva de un cliente se pierden sus datos. Como en el CSV cada cliente tiene una sola reserva, esto pasaría con los 1000.

4. Proceso de normalización

Partimos de la tabla única y la llevamos a tercera forma normal (3FN).

Paso	Cómo quedó
Origen	1 tabla de 29 columnas, una fila por reserva.
1FN	Ya cumple: cada celda tiene un solo valor y no hay grupos repetidos. La clave es `Booking_ID`, que no se repite.
2FN	Ya cumple: la clave es una sola columna, así que no puede haber dependencias parciales.
3FN	No cumple por las dependencias transitivas. Separamos país, ciudad, cliente y hotel.
Resultado	12 tablas con PK, FK y restricciones CHECK.

4.1 Primera y segunda forma normal

El CSV ya tiene valores atómicos y una clave sin repetidos, así que está en 1FN. Hay un solo caso que la rompería: las etiquetas que el alojamiento le pone a una reseña. Una reseña puede tener varias, y guardarlas juntas en una columna ("atencion, limpieza") no cumple la 1FN. Por eso van en su propia tabla, `resena_etiqueta`, con una fila por etiqueta.

La 2FN también se cumple, porque la clave es una sola columna. `resena_etiqueta` tiene clave compuesta (`booking_id`, `etiqueta`), pero no tiene otras columnas, así que tampoco tiene dependencias parciales.

4.2 Tercera forma normal

Las dependencias que encontramos son:

```
Booking_ID  → Customer_ID → Customer_Name, Customer_Age, Gender, Customer_City →
Customer_Country
Booking_ID  → Hotel_Name  → Destination_City → Destination_Country
```

Para sacarlas separamos:

- **PAIS** (nombre, region): una fila por país. La región no venía en el CSV y la agregamos (seis regiones) para agrupar las estadísticas.
- **CIUDAD** (id, nombre, pais_nombre): una sola tabla para las ciudades de los clientes y de los hoteles, 44 en total.
- **CLIENTE** (customer_id, nombre, edad, genero, ciudad_id).
- **HOTEL** (id, nombre, categoria_base, ciudad_id).
- **RESERVA** se queda solo con sus datos y las claves foráneas.

4.3 Catálogos

`Meal_Plan`, `Payment_Method`, `Transportation_Type` y `Booking_Status` tienen pocos valores que se repiten mil veces. No rompen la 3FN, pero los pasamos a tablas de catálogo (`regimen`, `medio_pago`, `transporte` y `estado_reserva`) para no repetir texto y evitar errores de tipeo. Quedaron 4 regímenes, 6 medios de pago, 3 transportes y 4 estados.

4.4 Columnas que sacamos y tablas opcionales

- Sacamos `Number_of_Travellers`, `Number_of_Nights` y `Cancellation_Status` porque se calculan.
- **CANCELACION** (id, booking_id, motivo) existe solo para las 141 reservas canceladas, así no hay que guardar "Not Applicable".
- **RESENA** (booking_id, puntajes, comentario, respuesta) existe solo para las 772 reservas completadas, así no hay que guardar puntajes en 0.
- **RESENA_ETIQUETA** (booking_id, etiqueta) guarda las etiquetas que carga el alojamiento.

4.5 Dónde quedó cada columna del CSV

Columna del CSV	Dónde quedó
Booking_ID, Booking_Date, Travel_Date, Return_Date	reserva
Customer_ID, Customer_Name, Customer_Age, Gender	cliente
Customer_City, Destination_City	ciudad (desde cliente.ciudad_id y hotel.ciudad_id)
Customer_Country, Destination_Country	pais (a través de ciudad)
Hotel_Name	hotel
Hotel_Rating	resena.puntaje_hotel y, como promedio, hotel.categoria_base
Number_of_Adults, Number_of_Children, Number_of_Rooms	reserva
Number_of_Travellers, Number_of_Nights	no se guardan, se calculan
Meal_Plan / Payment_Method / Transportation_Type / Booking_Status	regimen / medio_pago / transporte / estado_reserva
Cancellation_Status	no se guarda, sale del estado
Cancellation_Reason	cancelacion.motivo
Discount_Amount, Total_Trip_Cost	reserva
Customer_Rating, Customer_Review_Comment	resena.puntaje_viaje y resena.comentario

5. Diagrama Entidad-Relación

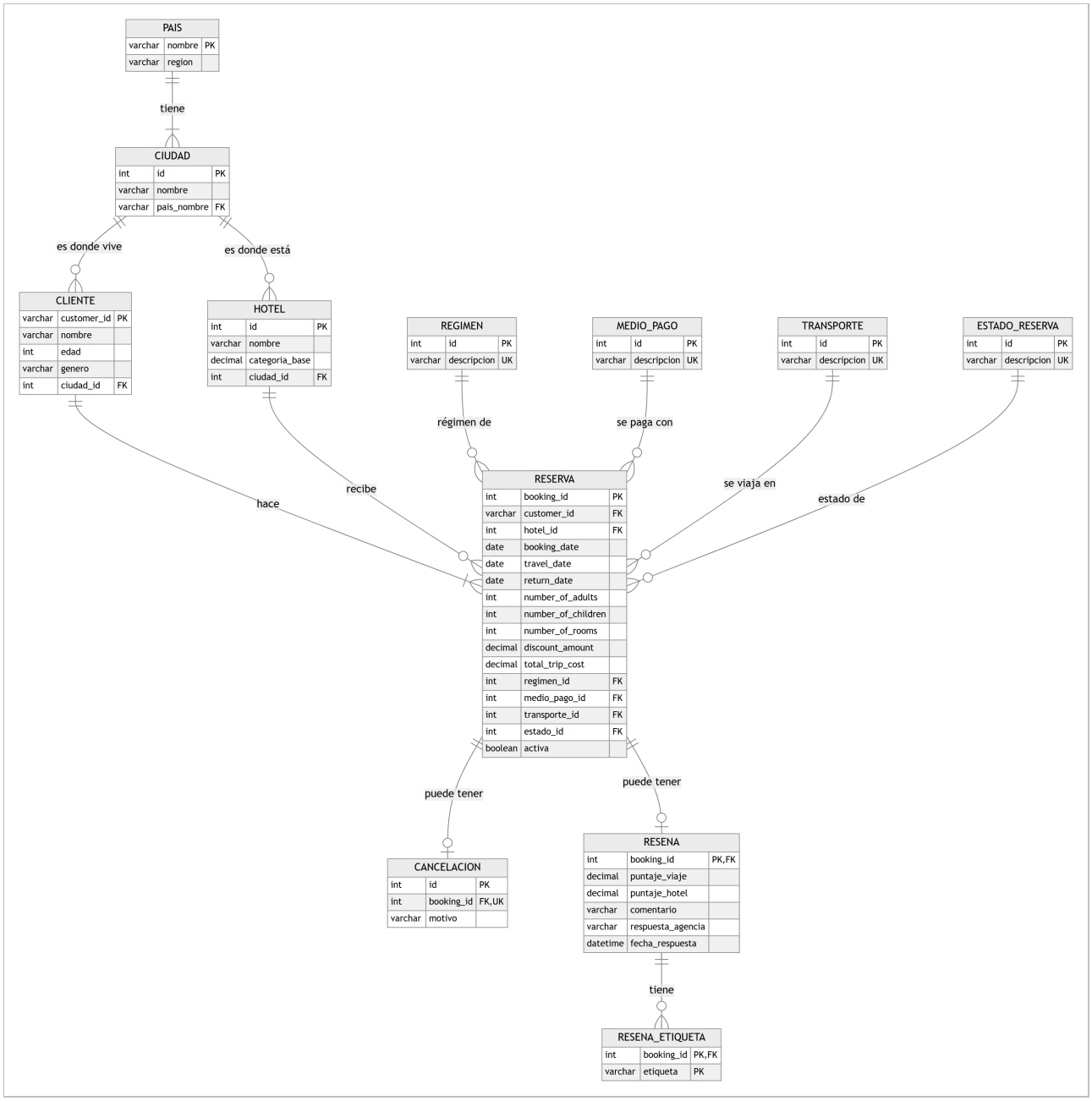


Figura 6. DER de la base reservas_viajes (también en documentacion/DER.png y DER.svg).

- Un país tiene una o más ciudades, y cada ciudad es de un solo país (1:N).
- En una ciudad pueden vivir varios clientes y puede haber varios hoteles (1:N).
- Un cliente hace una o más reservas y un hotel recibe muchas. RESERVA resuelve la relación N:M entre CLIENTE y HOTEL.
- Cada reserva tiene un régimen, un medio de pago, un transporte y un estado (N:1 con cada catálogo).

- Una reserva puede tener una cancelación y una reseña, o ninguna (1:0..1). En CANCELACION el `booking_id` es UNIQUE, y en RESENA el `booking_id` es a la vez PK y FK.
- Una reseña puede tener varias etiquetas (1:N).

6. Modelo relacional

El script `database/modelo_relacional.sql` crea la base `reservas_viajes` con las 12 tablas. `database/datos_iniciales.sql` carga los datos y lo genera nuestro script de carga (`database/cargar_datos.py`) a partir del CSV: primero verifica las dependencias y las reglas, y después arma los INSERT en el orden que piden las claves foráneas.

Tabla	Columnas (tipo)	PK	FK	Restricciones
pais	nombre VARCHAR(100), region VARCHAR(100)	nombre	-	region NOT NULL
ciudad	id INT, nombre VARCHAR(100), pais_nombre VARCHAR(100)	id	pais_nombre → pais	UNIQUE (nombre, pais_nombre)
cliente	customer_id VARCHAR(50), nombre VARCHAR(150), edad INT, genero VARCHAR(20), ciudad_id INT	customer_id	ciudad_id → ciudad	edad ≥ 18; genero 'Male' o 'Female'
hotel	id INT, nombre VARCHAR(150), categoria_base DECIMAL(3,1), ciudad_id INT	id	ciudad_id → ciudad	UNIQUE (nombre, ciudad_id); categoria_base entre 1 y 5
regimen, medio_pago, transporte, estado_reserva	id INT, descripcion VARCHAR(100)	id	-	descripcion UNIQUE
reserva	booking_id INT, customer_id VARCHAR(50), hotel_id INT, booking_date / travel_date / return_date DATE, number_of_adults / number_of_children / number_of_rooms INT, discount_amount / total_trip_cost DECIMAL(10,2), regimen_id / medio_pago_id / transporte_id / estado_id INT, activa BOOLEAN	booking_id	customer_id → cliente; hotel_id → hotel; regimen_id, medio_pago_id, transporte_id y estado_id → catálogos	fecha de reserva ≤ salida < regreso; adultos ≥ 1; niños ≥ 0; habitaciones entre 1 y la cantidad de viajeros; costo > 0; descuento entre 0 y el costo
cancelacion	id INT, booking_id INT, motivo VARCHAR(255)	id	booking_id → reserva	booking_id UNIQUE (una cancelación por reserva)
resena	booking_id INT, puntaje_viaje / puntaje_hotel DECIMAL(2,1), comentario VARCHAR(255), respuesta_agencia VARCHAR(500),	booking_id	booking_id → reserva	puntajes entre 1 y 5

	fecha_respuesta DATETIME			
resena_etiqueta	booking_id INT, etiqueta VARCHAR(40)	(booking_id, etiqueta)	booking_id → resena	-

Todas las claves foráneas usan la acción por defecto (RESTRICT): no se puede borrar una fila que otra tabla está usando.

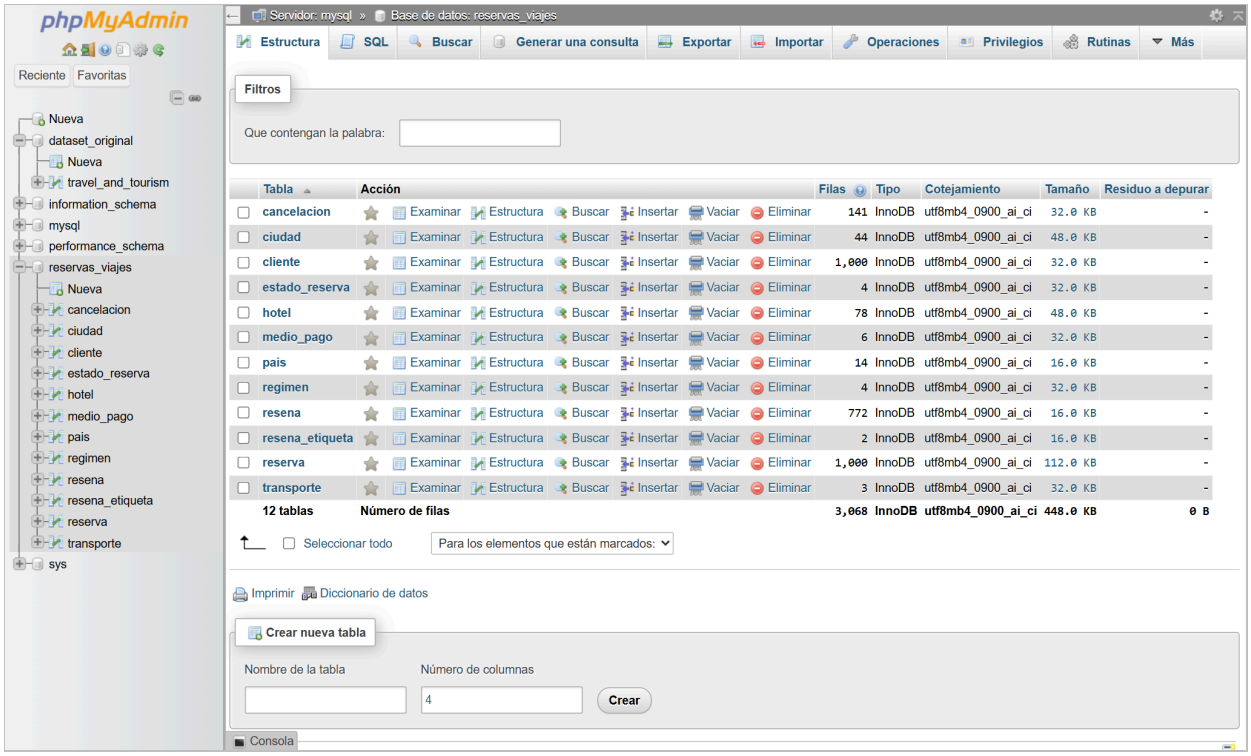


Figura 7. Las 12 tablas de reservas_viajes en phpMyAdmin, con los datos cargados. Las 2 etiquetas las cargamos probando la aplicación.

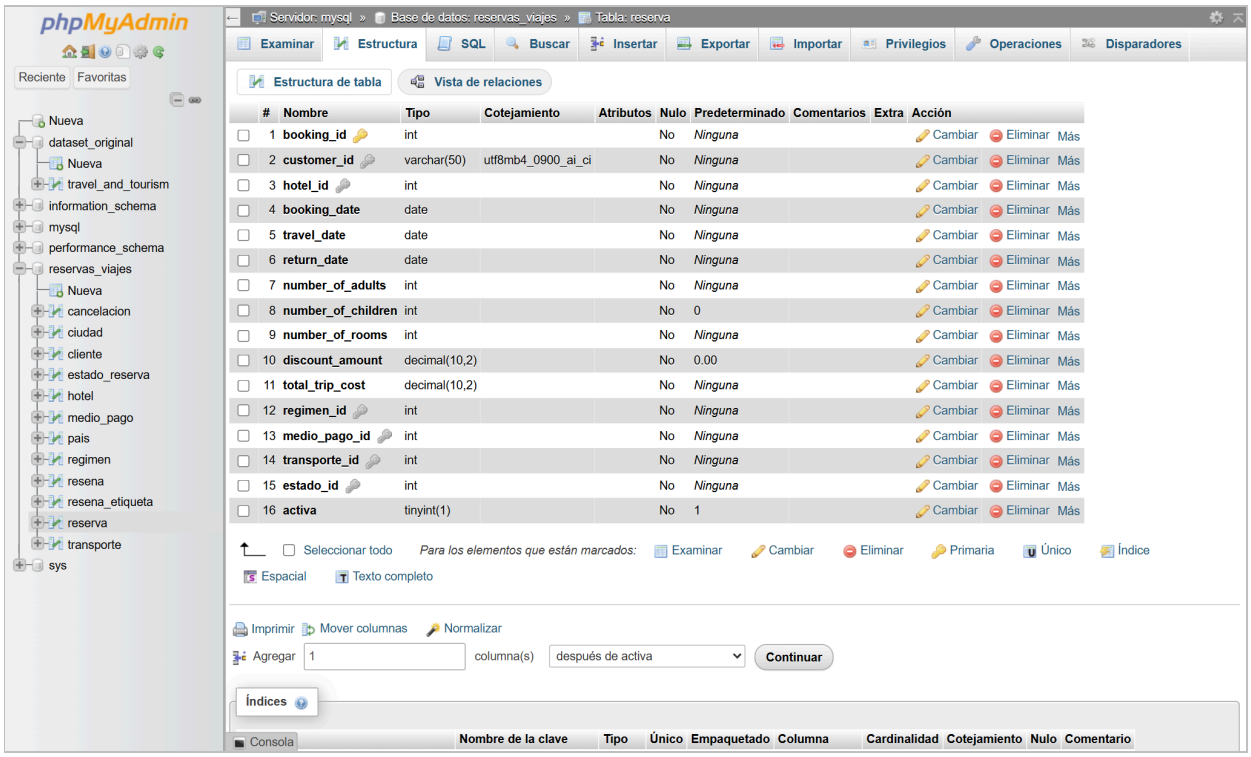


Figura 8. Estructura de la tabla reserva: tipos, nulos y valores por defecto.

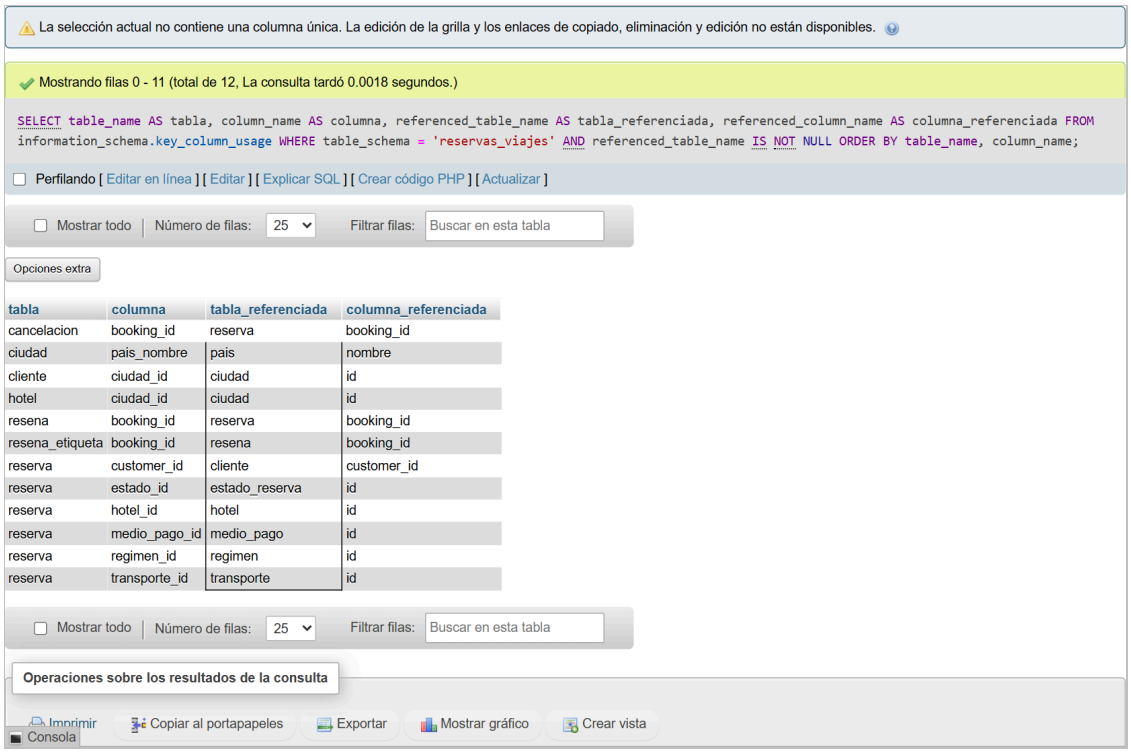


Figura 9. Las claves foráneas del modelo, consultadas en information_schema.

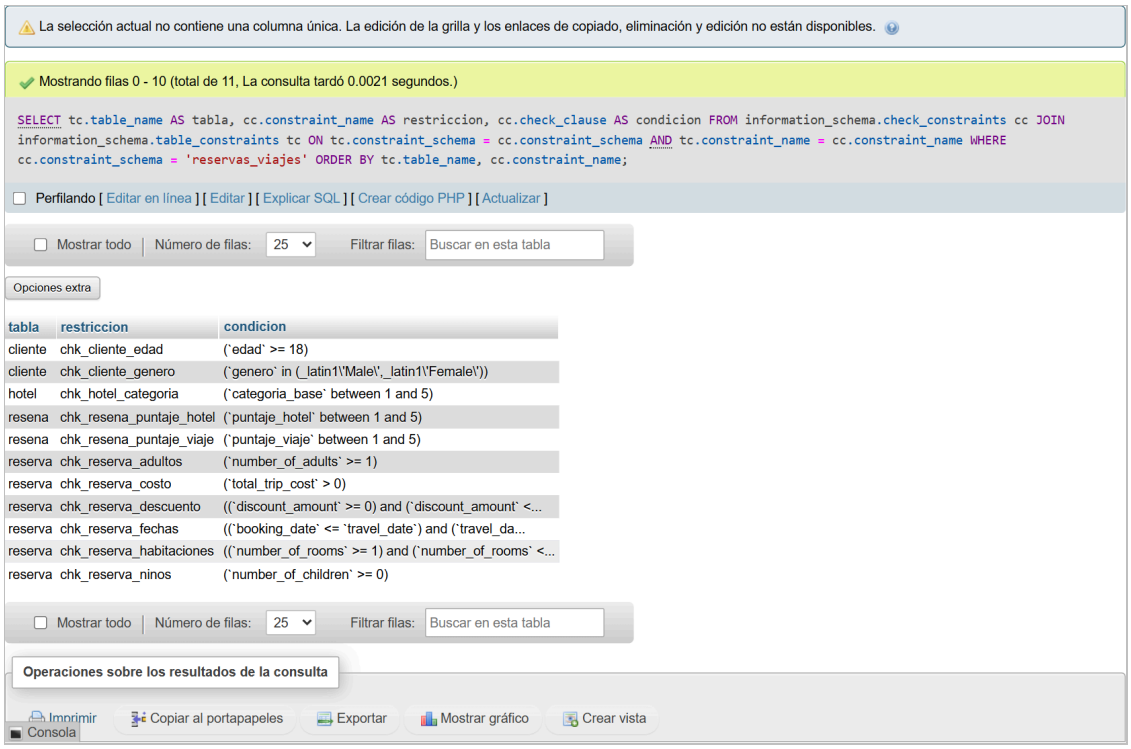


Figura 10. Las 11 restricciones CHECK del modelo.

6.1 Las restricciones funcionan

Probamos que la base rechace datos inválidos. Una reserva que vuelve antes de salir no cumple `chk_reserva_fechas` (Figura 11). Borrar una reserva que tiene una cancelación tampoco se puede, porque la clave foránea de `cancelacion` lo impide (Figura 12). Esto además nos ayuda con la regla de no borrar reservas.

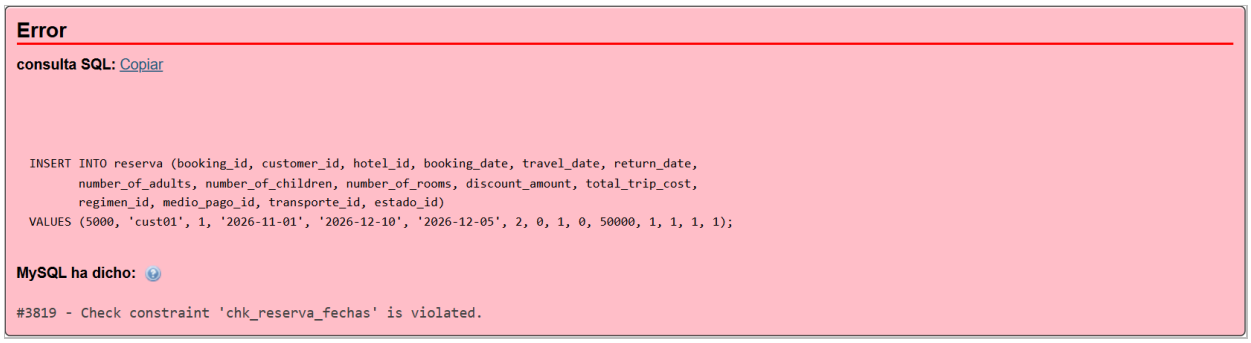


Figura 11. MySQL rechaza una reserva con la fecha de regreso anterior a la de salida.

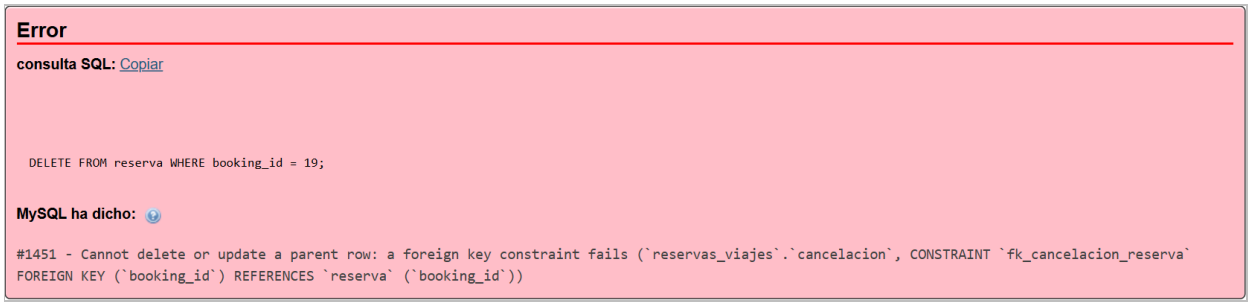


Figura 12. MySQL no deja borrar la reserva 19 porque tiene una cancelación.

7. Diseño de la solución NoSQL

Usamos **MongoDB**, una base de datos documental. La base `reservas_viajes` tiene dos colecciones: `reservas` y `resenas`. Los documentos los genera el mismo script de carga a partir del CSV (`database/scripts_nosql/reservas.json` y `resenas.json`) y se importan con `mongoimport`.

7.1 Colección reservas

Un documento por reserva, con el `booking_id` como `_id`. Los datos del cliente, del hotel y las fechas van adentro del documento, y los catálogos (régimen, medio de pago, transporte y estado) se guardan como texto. La cancelación aparece como subdocumento solo cuando la reserva se canceló.

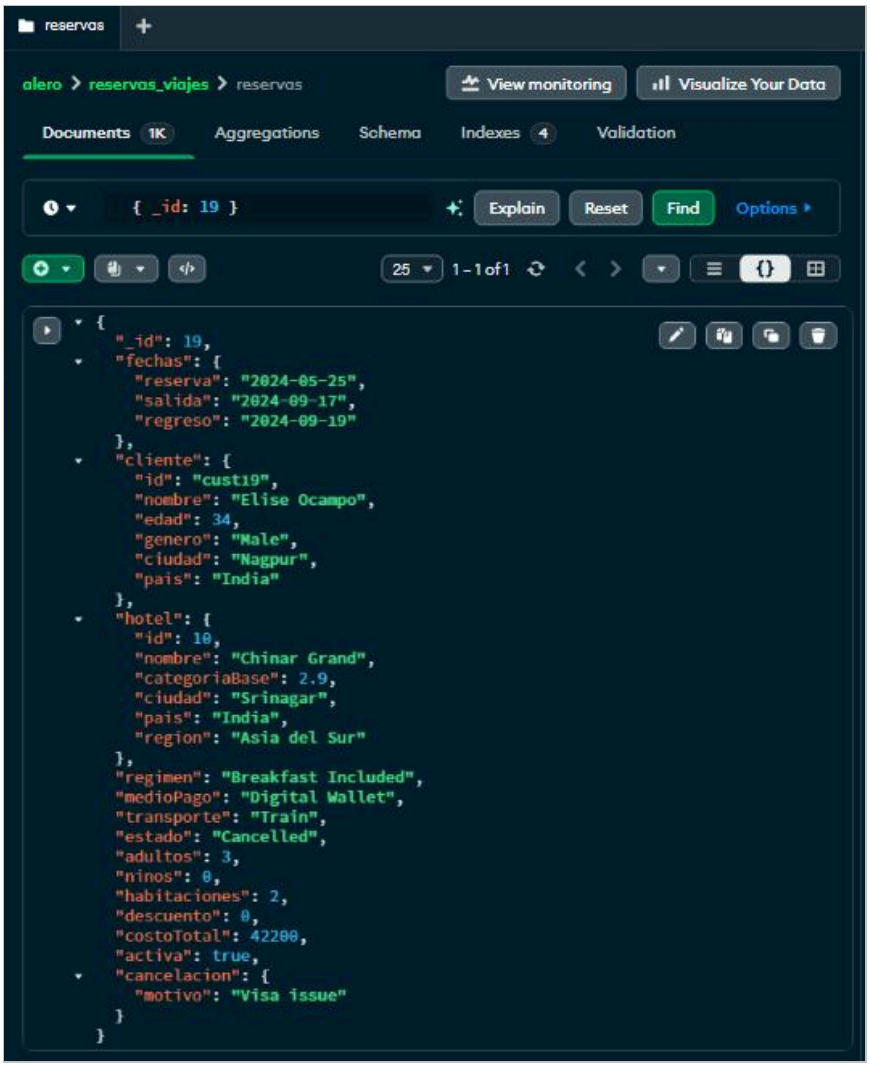


Figura 13. La reserva 19 en MongoDB Atlas: cliente, hotel y cancelación dentro del mismo documento.

7.2 Colección resenas

Un documento por reseña, solo para reservas completadas. Guarda el id de la reserva, el hotel y el destino copiados, los dos puntajes, el comentario, un array de etiquetas y un objeto opcional con la

respuesta del alojamiento. En los datos iniciales las etiquetas vienen vacías y no hay respuesta, porque las carga el alojamiento desde la aplicación.

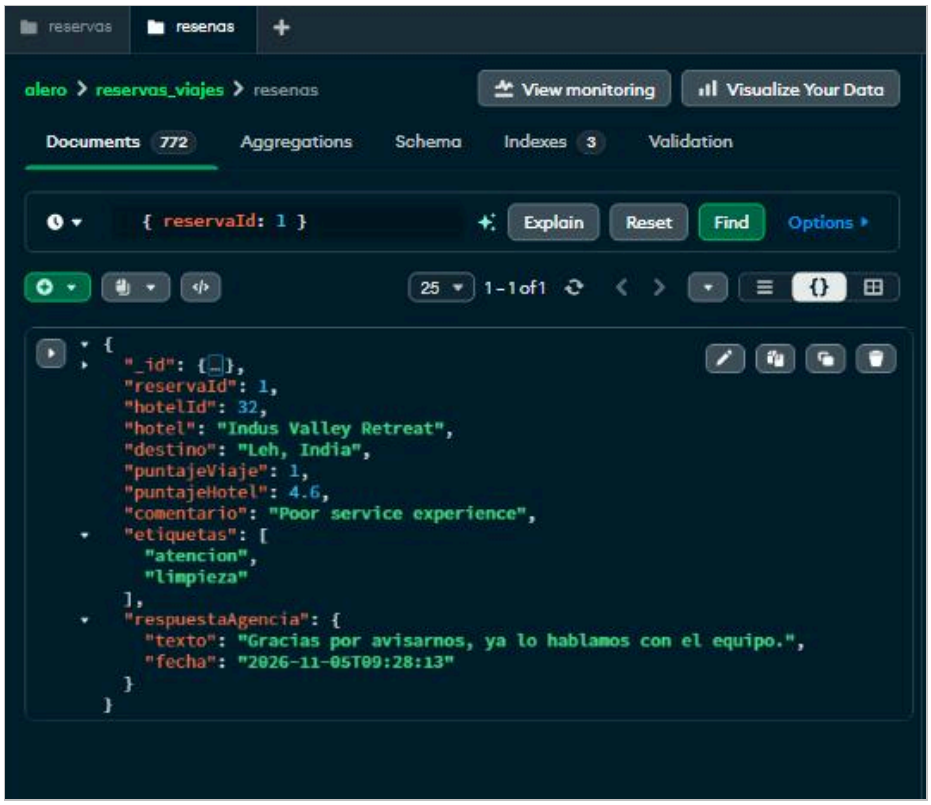


Figura 14. La reseña de la reserva 1, con dos etiquetas y la respuesta del alojamiento cargadas desde la aplicación.

7.3 Validación e índices

MongoDB no tiene claves foráneas ni restricciones CHECK. Para no aceptar cualquier cosa, el script `database/scripts_nosql/indices.js` le pone a cada colección un validador `$jsonSchema` con los campos obligatorios, los tipos, los estados permitidos, la edad mínima de 18, el formato de las fechas (AAAA-MM-DD) y los puntajes entre 1 y 5. Si intentamos insertar a mano una reseña con puntaje 9, la base la rechaza (Figura 16). El resto de la integridad la controla el backend: antes de guardar una reseña verifica que la reserva exista, que esté completada y que no tenga otra reseña.

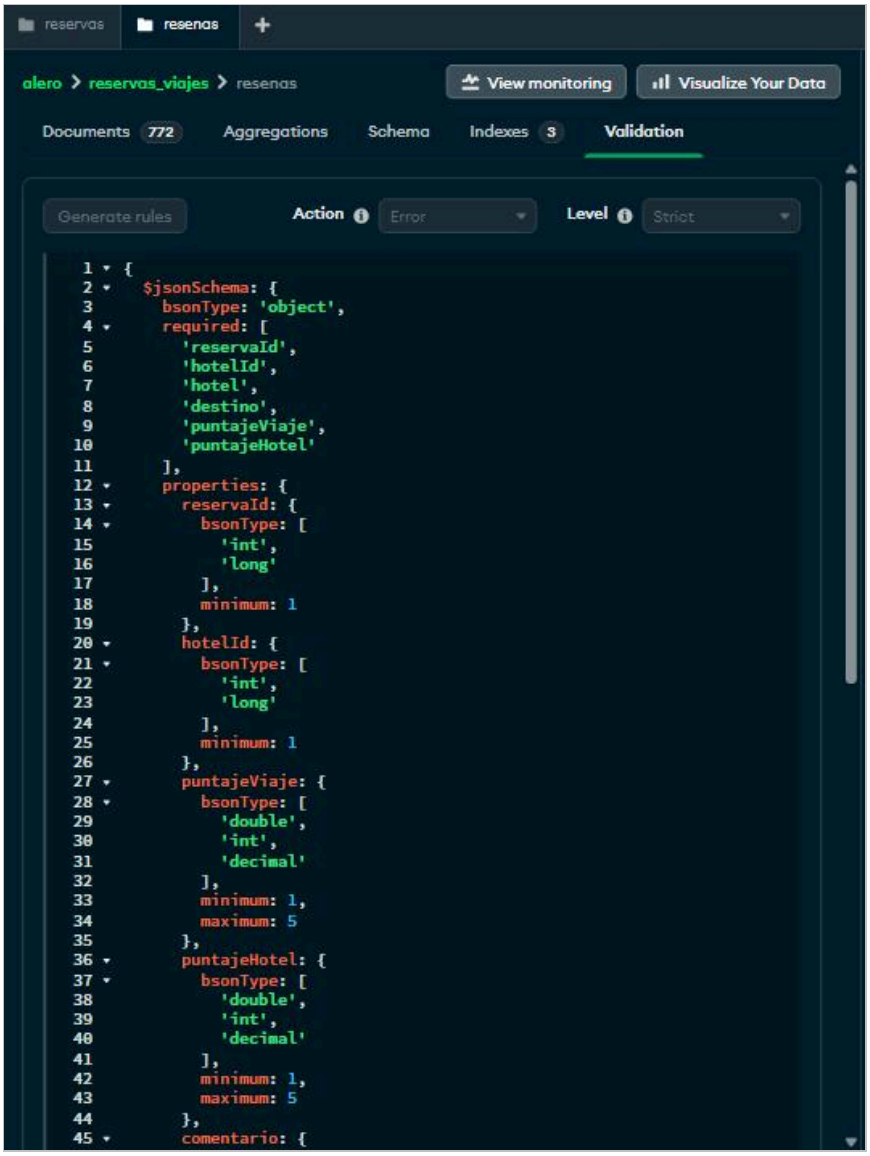


Figura 15. Parte del validador de la colección resenas (acción Error, nivel Strict).

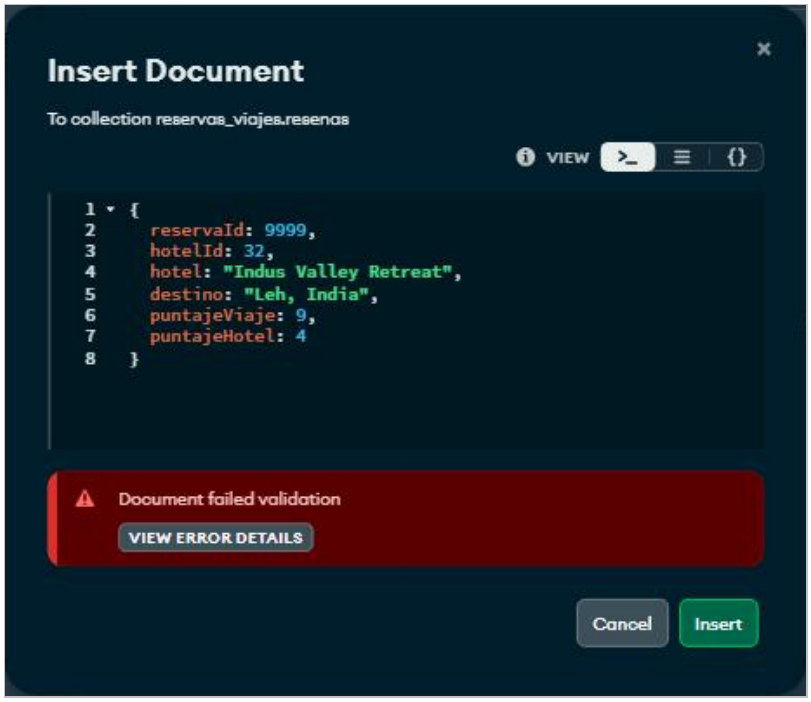


Figura 16. MongoDB rechaza una reseña con puntajeViaje 9.

Los índices son:

- **reservas:** `estado`, `hotel.id` y `hotel.pais`, que son los filtros del listado y los campos por los que agrupan las estadísticas.
- **resenas:** `reservaId` único (una reseña por reserva, igual que la PK de resena en MySQL) y `hotelId` para listar y promediar por alojamiento.

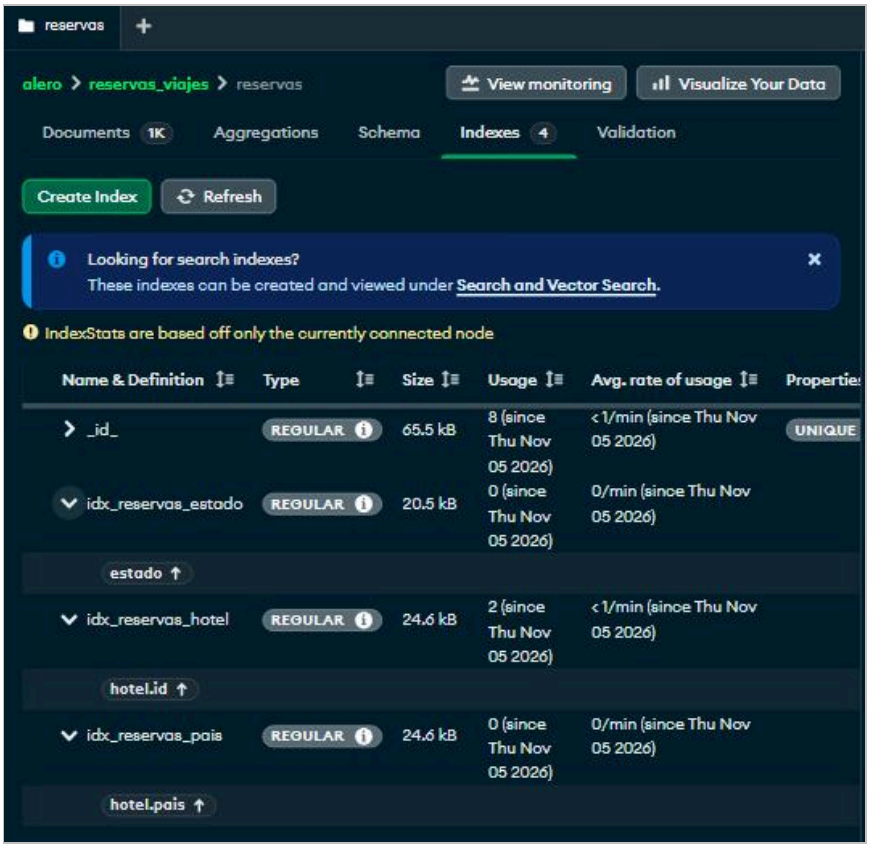


Figura 17. Índices de la colección reservas en Atlas.

8. Justificación de la utilización de NoSQL

Qué guardaríamos en NoSQL. La parte del sistema que más se beneficia son las **reseñas**. Las etiquetas son una lista de largo variable: en MongoDB son un array dentro del documento y en MySQL necesitan una tabla aparte y un JOIN. La respuesta del alojamiento es opcional: en MongoDB el objeto aparece solo cuando existe, y en MySQL son dos columnas que quedan en NULL en casi todas las filas. Además, la consulta más común (las reseñas de un alojamiento con su promedio) se resuelve con la colección sola, porque el hotel ya está copiado en cada reseña. También nos sirve tener la **reserva completa** en un documento, porque se lee de una vez, sin los 9 JOIN que hacen falta en MySQL (Figura 21).

Qué tipo de base. Una base documental (MongoDB), porque los datos se usan como fichas completas (una reserva, una reseña) y los documentos JSON se parecen a cómo los muestra la aplicación.

Ventajas frente al modelo relacional:

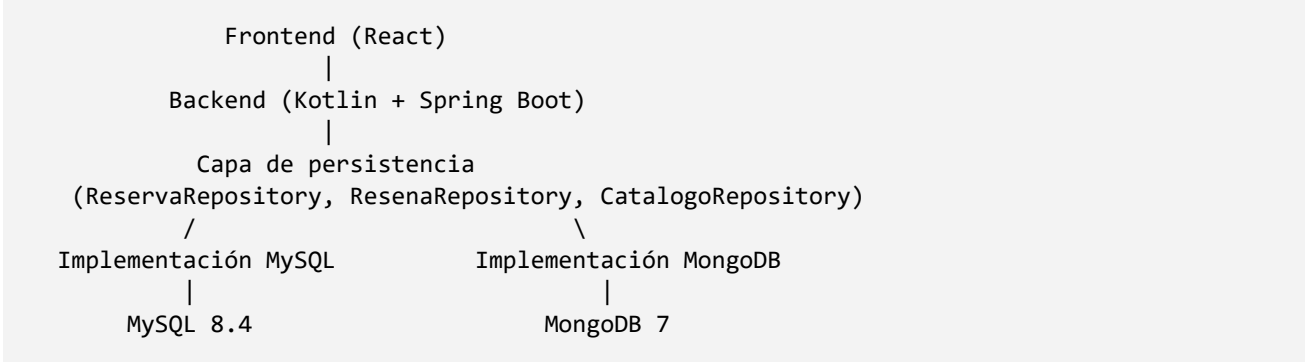
- Se lee todo en un solo acceso, sin JOIN.
- El esquema es flexible para los datos opcionales (cancelación, respuesta) y las listas (etiquetas).
- Las estadísticas por hotel o por país se hacen con un `aggregate` sobre una sola colección.

Desventajas y limitaciones:

- Hay datos duplicados: el cliente y el hotel están copiados en cada reserva y en cada reseña. Si un hotel cambia de nombre hay que actualizar todos sus documentos, que es justo la anomalía que la 3FN evita en MySQL.
- No hay claves foráneas: la integridad depende de los validadores y del backend. Si alguien escribe en la base sin pasar por el backend, nada impide una reseña de una reserva que no existe.
- Las operaciones que tocan dos colecciones no tienen la misma garantía que una transacción de MySQL.

Cómo conviven los dos motores. En el trabajo los dos motores funcionan al mismo tiempo detrás del mismo backend. Cada uno tiene su propia copia de los datos, cargada desde el mismo CSV, y la aplicación puede trabajar con cualquiera de los dos. Así cumplimos con el backend polimórfico (cada operación tiene una implementación para MySQL y otra para MongoDB) y podemos comparar cómo se resuelve lo mismo en cada modelo. Como permite la consigna, la estructura no es la misma: 12 tablas en MySQL y 2 colecciones en MongoDB.

9. Arquitectura general de la aplicación



El frontend nunca accede a las bases: todo pasa por la API del backend. En la computadora se levanta todo con Docker Compose (MySQL, phpMyAdmin, MongoDB, backend y frontend). También publicamos una versión en la nube con planes gratuitos: la aplicación y la API en Render, MySQL en Aiven y MongoDB en Atlas.

10. Explicación del backend polimórfico

Cada repositorio es una interfaz con dos implementaciones, una por motor. El resto del backend (los servicios y las reglas de negocio) usa solo la interfaz y no sabe qué base hay atrás.

Interfaz	Implementación MySQL	Implementación MongoDB
ReservaRepository	MySqlReservaRepository (Spring Data JPA)	MongoReservaRepository (MongoTemplate)
ResenaRepository	MySqlResenaRepository	MongoResenaRepository
CatalogoRepository	MySqlCatalogoRepository	MongoCatalogoRepository

El motor se elige en cada pedido: el frontend manda el header `X-DB-Engine` con `mysql` o `mongo` según el selector de la pantalla. Si un pedido no lo trae, se usa el motor por defecto de la

configuración (MySQL). Cada respuesta devuelve el header `X-DB-Engine-Used` con el motor que se usó.

Operaciones con los dos motores:

- **Reservas:** alta, consulta individual, listado con filtros, modificación, baja lógica, cancelación con motivo y registro de la salida.
- **Reseñas:** alta, consulta, listado por alojamiento, respuesta y etiquetas.
- **Consultas complejas:** facturación y tasa de cancelación por país, y reservas, cancelaciones y puntajes por alojamiento.

11. Tecnologías utilizadas

Parte	Tecnología
Base relacional	MySQL 8.4, administrada con phpMyAdmin
Base NoSQL	MongoDB 7 (MongoDB Atlas en la nube)
Carga de datos	Python con pandas (`cargar_datos.py`), mongoimport y mongosh
Backend	Kotlin con Spring Boot (Spring Data JPA y MongoTemplate)
Frontend	React con Vite y TypeScript
Entorno	Docker Compose
Versión en la nube	Render, Aiven (MySQL) y MongoDB Atlas
Código	Git y GitHub

12. Instrucciones para ejecutar el proyecto

Con Docker abierto, en la carpeta del proyecto:

```
docker compose up --build
```

Eso genera los datos desde el CSV, crea las tablas y carga MySQL, importa las colecciones de MongoDB con sus validadores e índices, y levanta el backend y el frontend:

- Aplicación: http://localhost:5173
- API: http://localhost:9000
- phpMyAdmin: http://localhost:8081

Sin instalar nada se puede usar la versión publicada: https://alero-tp-bd.onrender.com. Como es un plan gratuito, la API se duerme a los 15 minutos sin uso y el primer pedido tarda cerca de un minuto. El `README.md` tiene las instrucciones completas, incluso cómo levantarlo sin Docker.

13. Ejemplos de funcionamiento

13.1 La aplicación

La pantalla de reservas lista las reservas activas, el historial y las dadas de baja. Abajo a la izquierda está el selector de motor, que muestra cuál respondió el último pedido.

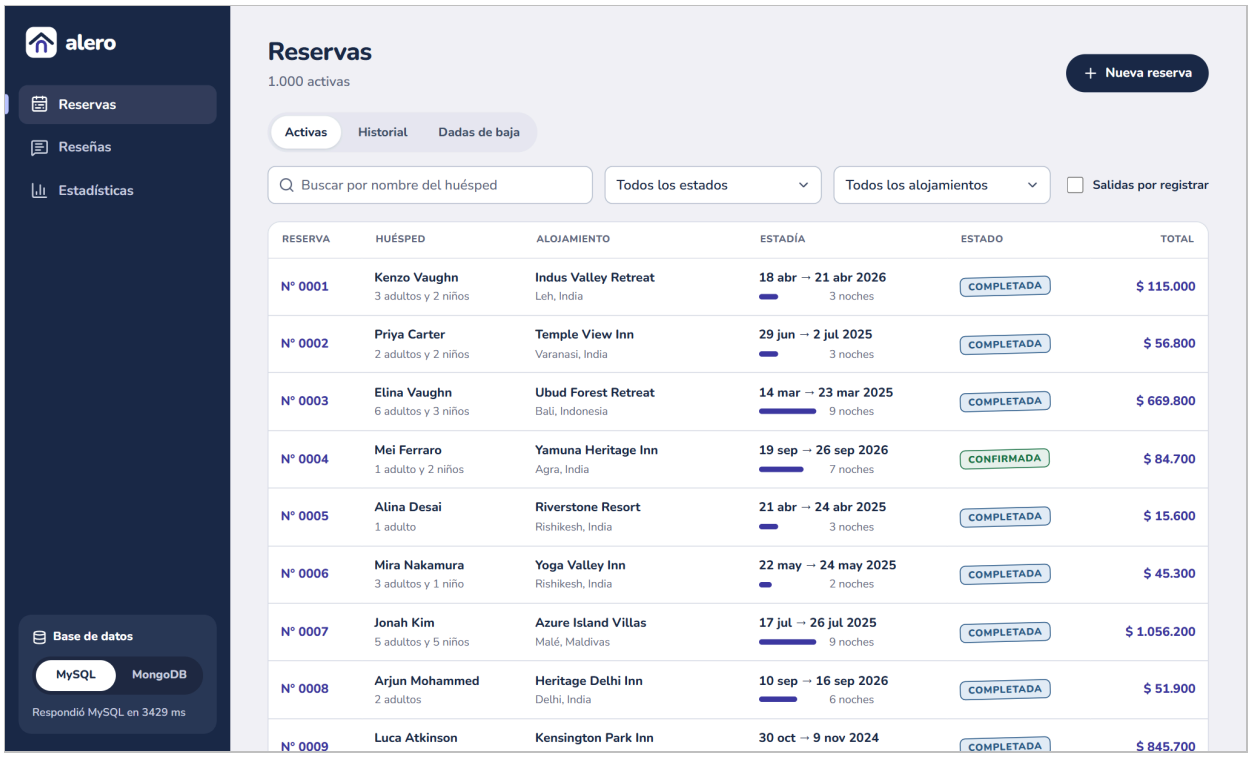


Figura 18. Listado de reservas trabajando con MySQL.

13.2 Una consulta compleja en los dos motores

Facturación y tasa de cancelación por país de destino. En MySQL hace falta unir reserva, hotel, ciudad y país, y contar las cancelaciones con un LEFT JOIN:

```
SELECT p.nombre AS pais, p.region,
       COUNT(*) AS reservas,
       COUNT(c.booking_id) AS canceladas,
       SUM(CASE WHEN c.booking_id IS NULL THEN r.total_trip_cost ELSE 0 END) AS
facturacion
  FROM reserva r
  JOIN hotel h   ON h.id = r.hotel_id
  JOIN ciudad ci ON ci.id = h.ciudad_id
  JOIN pais p    ON p.nombre = ci.pais_nombre
  LEFT JOIN cancelacion c ON c.booking_id = r.booking_id
 WHERE r.activa = TRUE
 GROUP BY p.nombre, p.region
 ORDER BY facturacion DESC;
```

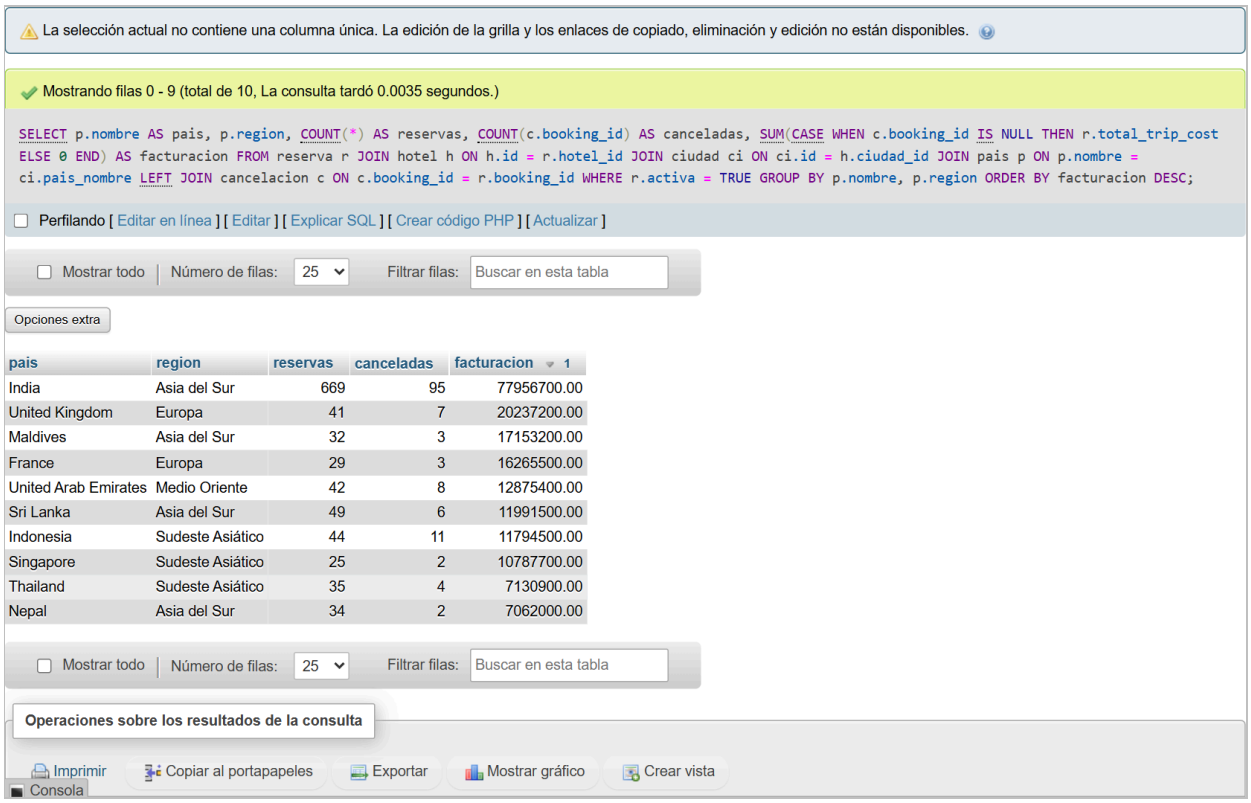


Figura 19. La consulta en MySQL desde phpMyAdmin.

En MongoDB es un `aggregate` sobre la colección reservas, sin JOIN, porque el país del hotel está en el documento:

```
db.reservas.aggregate([
  { $match: { activa: true } },
  { $group: {
      _id: "$hotel.pais",
      reservas: { $sum: 1 },
      canceladas: { $sum: { $cond: [{ $eq: ["$estado", "Cancelled"] }, 1, 0] } },
      facturacion: { $sum: { $cond: [{ $eq: ["$estado", "Cancelled"] }, 0, "$costoTotal"]
} }
  } },
  { $sort: { facturacion: -1 } }
])
```

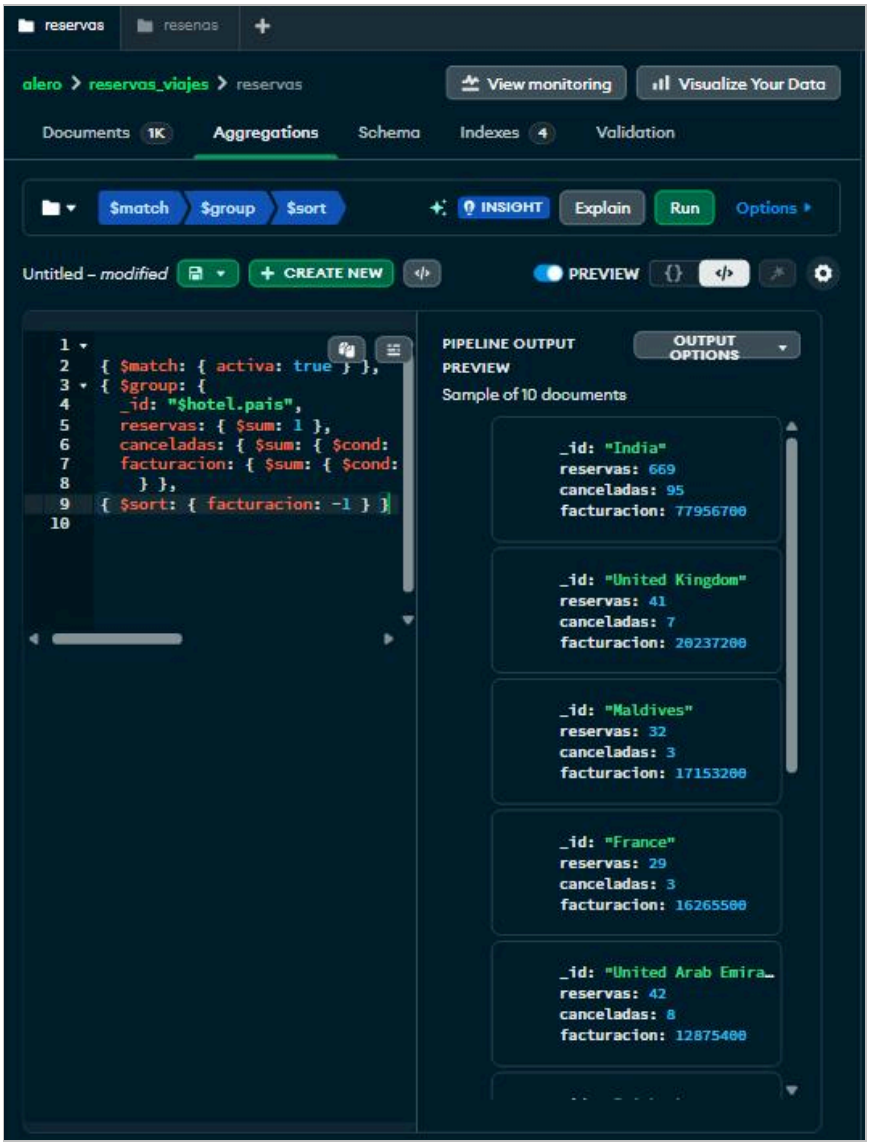


Figura 20. La misma consulta en MongoDB Atlas. India: 669 reservas, 95 canceladas y 77.956.700 de facturación, igual que en MySQL.

13.3 Leer una reserva completa

Para mostrar una reserva con todos sus datos, en MySQL hay que unir nueve tablas. En MongoDB es un solo documento (Figura 13).



Figura 21. La reserva 19 armada en MySQL con 9 JOIN.

13.4 Comparación desde la aplicación

En la pantalla de estadísticas, el botón "Comparar los dos motores" corre las mismas consultas en MySQL y en MongoDB, muestra cuánto tardó cada uno y verifica que den los mismos números.

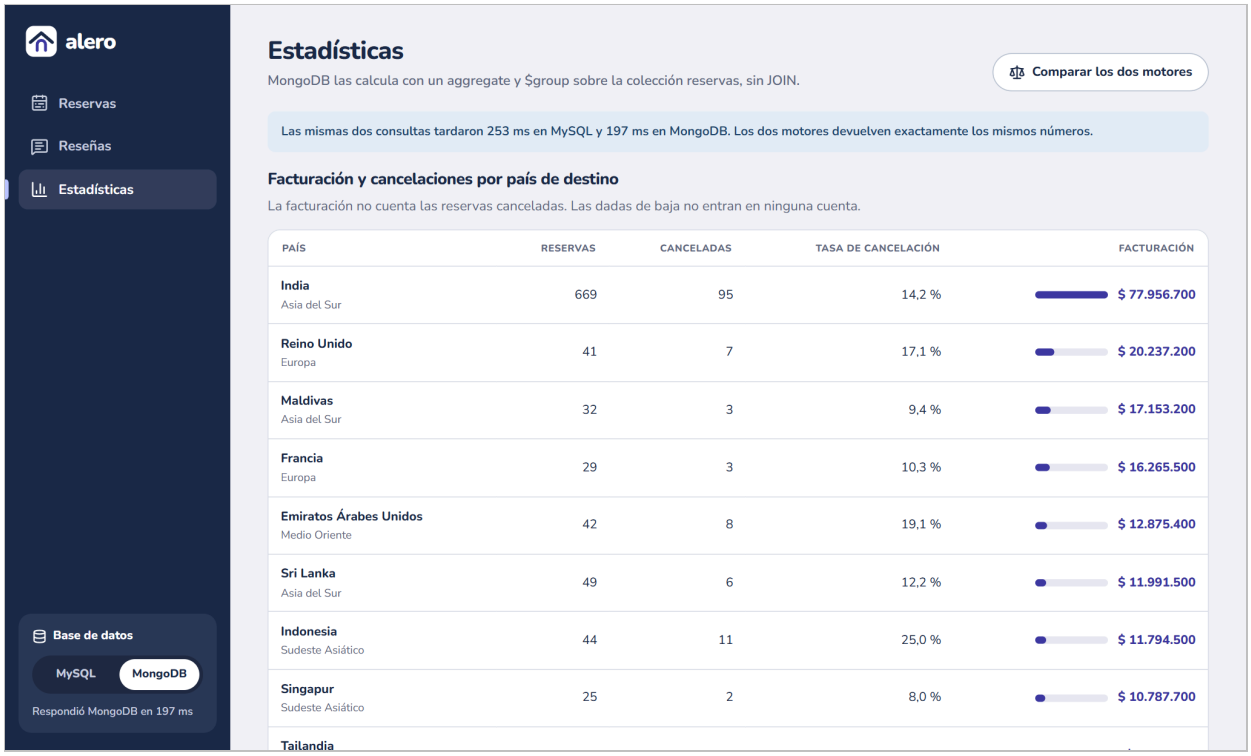


Figura 22. Estadísticas con MongoDB y la comparación de los dos motores en la versión publicada.

14. Comparación entre ambas implementaciones

Aspecto	MySQL	MongoDB
Modelo	12 tablas en 3FN	2 colecciones con datos embebidos
Leer una reserva	JOIN con cliente, ciudades, hotel, catálogos y cancelación	un solo documento
Integridad	PK, FK, UNIQUE y CHECK en la base	validador $jsonSchema, índice único y controles del backend
Cancelar	UPDATE de la reserva e INSERT en cancelacion, en una transacción	una sola actualización del documento
Etiquetas y respuesta	tabla resena_etiqueta y columnas que quedan en NULL	array y objeto opcional
Estadísticas	JOIN y GROUP BY	aggregate con $match y $group
Datos repetidos	ninguno	cliente y hotel copiados en cada documento

Tiempos que medimos desde la aplicación, con 1000 reservas:

Operación	MySQL	MongoDB
Listado de 20 reservas (en la computadora)	22 a 26 ms	10 a 11 ms
Detalle de una reserva (en la computadora)	14 a 17 ms	6 ms
Estadística por país (en la computadora)	8 ms	8 ms
Las dos estadísticas juntas (versión en la nube)	253 ms	197 ms

Con tan pocos datos las diferencias son de milisegundos, pero MongoDB lee más rápido justo donde el modelo relacional necesita varios JOIN. En la nube los tiempos son más altos porque incluyen la red. A cambio, MySQL garantiza la integridad en la propia base, y en MongoDB depende de que todos escriban a través del backend.

15. Problemas encontrados y decisiones tomadas

- **El puntaje del hotel cambia en cada reserva.** Lo guardamos como puntaje de la reseña y para el hotel usamos el promedio (`categoria_base`).
- **Valores "Not Applicable" y puntajes en 0.** Los reemplazamos por tablas opcionales (cancelacion y resena) en vez de guardar valores inventados.
- **La región no venía en el CSV.** La agregamos en el script de carga con seis regiones (Asia del Sur, Sudeste Asiático, Medio Oriente, Europa, América del Norte y Oceanía).
- **Cambios respecto de la primera versión del modelo (10 tablas).** Agregamos `number_of_adults` y `number_of_children`, que hacen falta para calcular los viajeros. También `transporte_id`, porque el catálogo transporte no lo usaba ninguna tabla, y `activa` para la baja lógica. Y sumamos `resena` y `resena_etiqueta` para que las reseñas también estén en MySQL y la aplicación pueda usar cualquiera de los dos motores.
- **Motivo de cancelación.** En la propuesta inicial pensamos un catálogo MOTIVO, pero quedó como texto en cancelacion, porque al cancelar el alojamiento puede escribir un motivo que no está en el CSV.

- **La reseña en MongoDB.** En la propuesta iba dentro del documento de la reserva. La pasamos a su propia colección porque se escribe en otro momento (después de la estadía), la responde y la etiqueta el alojamiento, y la consulta más común, las reseñas de un alojamiento, no necesita el resto de la reserva.
- **Los dos motores con todos los datos.** Al principio pensamos guardar las reservas en MySQL y las reseñas solo en MongoDB. Lo cambiamos para cumplir con el backend polimórfico, que pide que la misma operación se pueda hacer con cualquiera de los dos, y para poder comparar las dos implementaciones con las mismas consultas.
- **Integridad en MongoDB.** Como no hay claves foráneas, usamos validadores, un índice único en reservaId y controles en el backend.
- **No borrar reservas.** Usamos baja lógica (`activa`). Además, las FK con RESTRICT no dejan borrar una reserva que tiene cancelación o reseña (Figura 12).
- **Un error del validador.** Para pedir que el costo sea mayor que cero pusimos `exclusiveMinimum: 0` y MongoDB no lo aceptaba. En `$jsonSchema` se escribe `minimum: 0` junto con `exclusiveMinimum: true`.
- **Carga del CSV en MySQL.** Al cargar el CSV para analizarlo, `LOAD DATA` no cargaba ninguna fila porque le pusimos el fin de línea de Windows (`\r\n`) y el archivo usa `\n`.

16. Conclusiones

- Normalizar nos sacó las repeticiones y los problemas de actualización, inserción y eliminación: cada dato se guarda una sola vez, y con PK, FK y CHECK la propia base rechaza los datos inválidos.
- El costo de normalizar es que para armar una reserva completa hacen falta varios JOIN. MongoDB la devuelve en un solo documento y fue más rápido en las lecturas, aunque con 1000 reservas la diferencia es de milisegundos.
- MongoDB se adapta mejor a los datos opcionales y a las listas, como las reseñas con etiquetas y respuesta. A cambio, la integridad depende de los validadores y del backend, y los datos copiados hay que mantenerlos iguales.
- Las dos implementaciones dieron los mismos números en todas las consultas, y eso nos sirvió para comprobar que la carga y las consultas estaban bien.
- No hay un motor mejor en general. Para el núcleo de reservas, con muchas relaciones y reglas, nos quedamos con el modelo relacional. Para las reseñas, que tienen datos que cambian de forma, el modelo documental. Con un backend que usa interfaces comunes, los dos pueden convivir en la misma aplicación.

Anexo: archivos de la entrega

```
/backend                      backend en Kotlin + Spring Boot
/frontend                     frontend en React
/database
    modelo_relacional.sql     crea la base y las 12 tablas
    datos_iniciales.sql       carga los datos
    analisis_dataset.sql      CSV en una tabla y consultas del análisis
    cargar_datos.py           genera los datos desde el CSV
    /scripts_nosql            reservas.json, resenas.json e indices.js
/documentacion
    informe.pdf               este informe
    DER.png y DER.svg         diagrama entidad-relación
    /capturas                 capturas del informe
README.md                     cómo levantar y probar el proyecto
```